COMP S382F/4820SEF

Unit 4 K-nearest neighbours

(2026 Spring term)

Lecturer and Course Coordinator:
Dr. Alin Yalin Liu (ylliu@hkmu.edu.hk)

You can call me
"Missy", "Alin",
or "Dr. Liu"

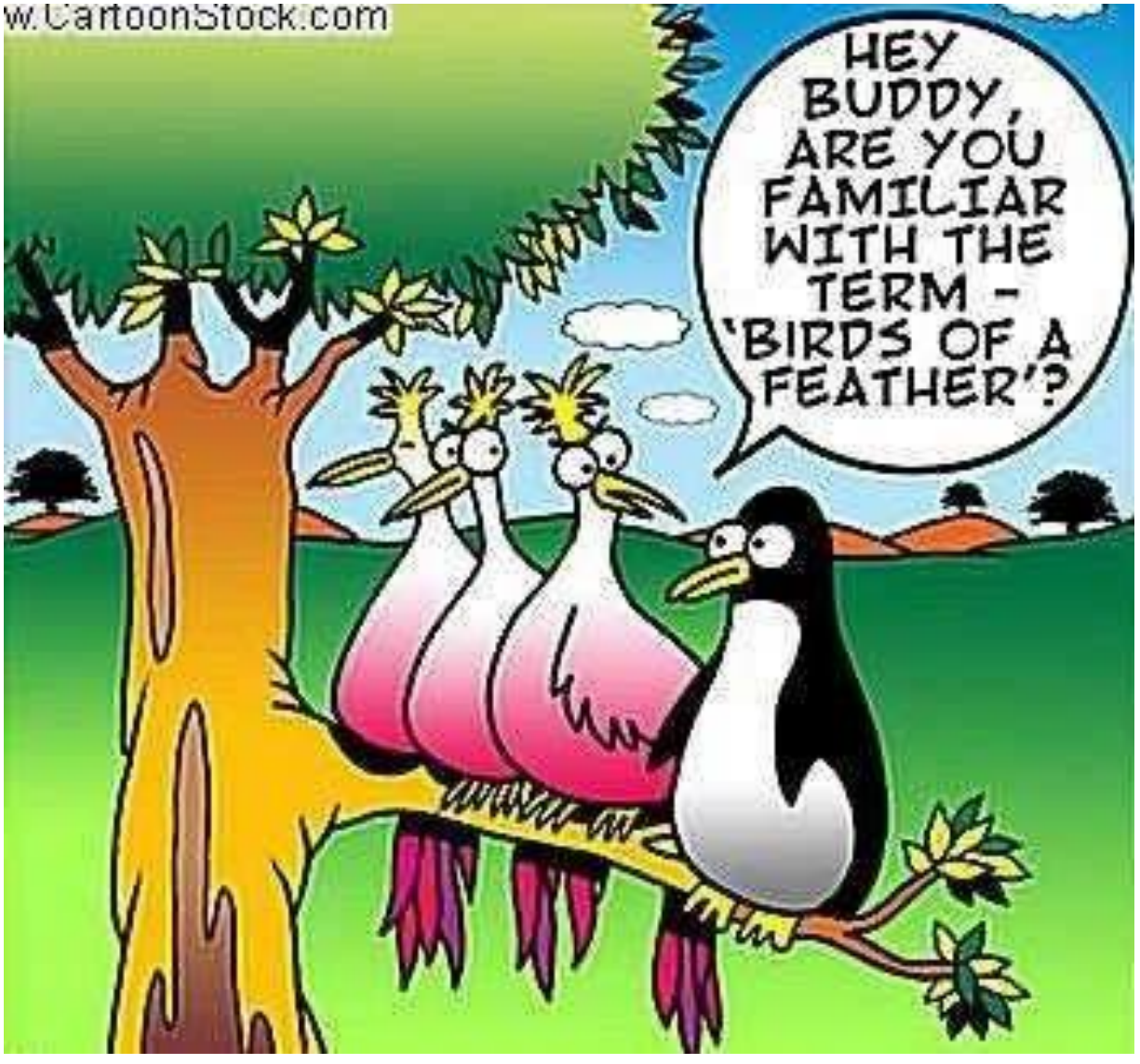
Contents

1. K-nearest neighbours
2. *k*-NN worked examples
3. Distance metrics
4. Characteristics of *k*-NN

1. K-nearest neighbours (*k*-NN)

"Birds of a feather flock together."

"Similar records (data objects) congregate in a neighborhood in n-dimensional space, with the same target class labels."



"Birds of a feather flock together."

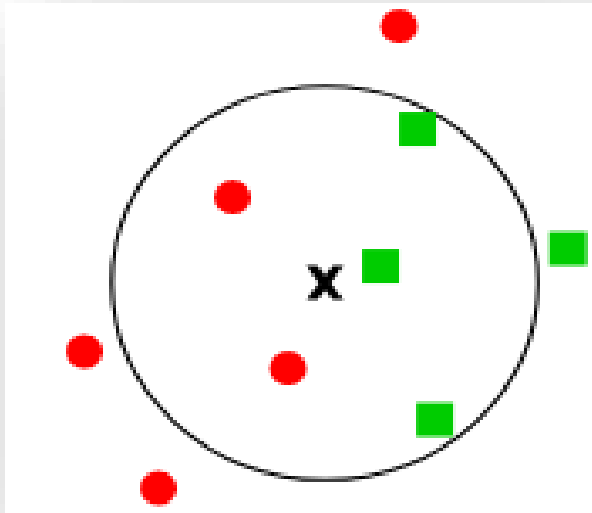
When people that act the same, hang out together.

People that have the same morals often tend to group.

K-nearest neighbours

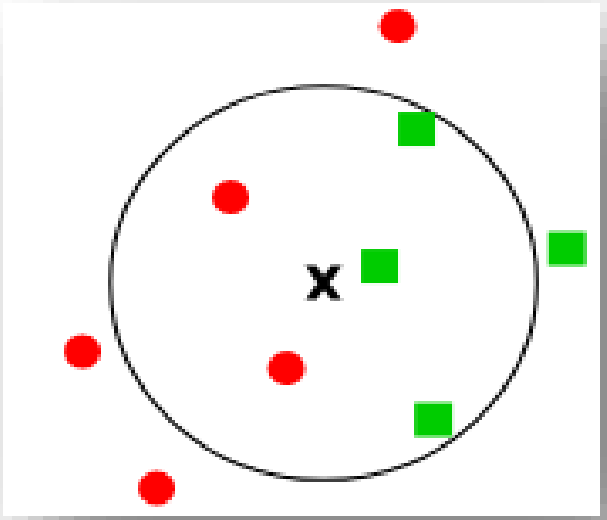
KNN, or *k*-NN, is a non-parametric, **supervised learning** algorithm, which makes use of **similar examples** to predict the label of an unlabelled example (*query point*).

- **Similar examples** are those that are nearest to *the query point*, i.e. with the shortest distances.



How *k*-NN works

1. Identify ***k* nearest neighbours**
   - Calculate the distance between the query point and all data points in the training dataset.
   - Select the k-nearest data points with the smallest distances to the new data point.



Dataset	Distance to the query point	Rank by minimum distance
#1	4.12	4
#2	4.00	3
#3	2.24	1
#4	3.00	2

Distance calculation

The Euclidean distance is most often used

- For two n-feature examples $x = (x_1, x_2, \ldots, x_n)$ and $y = (y_1, y_2, \ldots, y_n)$

$$\text{Distance}(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- In 2D, distance between $x = (x_1, x_2)$ and $y = (y_1, y_2)$ is

$$\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

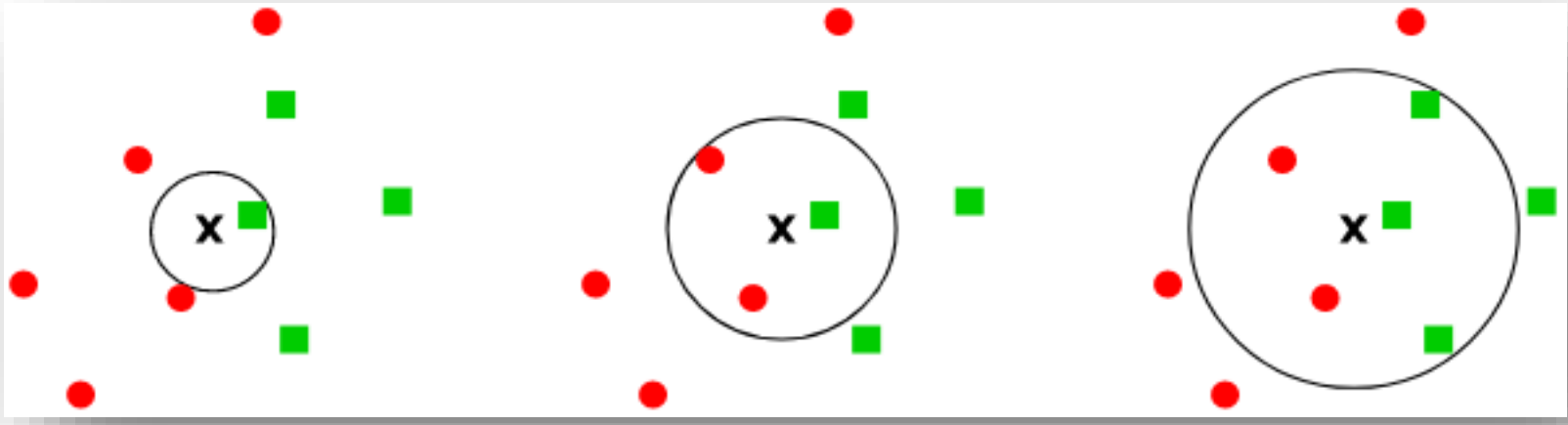
How *k*-NN works

1. Identify ***k* nearest neighbours**
2. Take **the average of the *k* neighbours' labels**

Majority voting

- For classification, the average is the most frequent class
- For regression, the average is the mean value

The value of *k* affects how kNN makes predictions!



E.g. a classification illustration: is 'X' a circle or a square? For *k* of 1, 3, 5

Applications of *k*-NN

- **Recommendation Systems**: E.g., Netflix or Amazon. KNN observes at user behavior and finds similar users. If user A and user B have similar preferences, KNN might recommend movies that user A liked to user B.

- **Spam Detection**: By comparing the features of a new email with those of previously labeled spam and non-spam emails, KNN can predict whether a new email is spam or not.

- **Customer Segmentation**: By comparing new customers to existing customers, KNN can easily group customers into segments with similar choices and preferences. This helps businesses target the right customers with right products or advertisements.

- **Speech Recognition**: KNN is often used in speech recognition systems to transcribe spoken words into text. The algorithm compares the features of the spoken input with those of known speech patterns. It then predicts the most likely word or command based on the closest matches.

Ref: *https://www.geeksforgeeks.org/k-nearest-neighbours/*

2. *k*-NN worked examples

Classification example

Consider the amounts of sugar and carbon dioxide (CO_2) for making a good soft drink

Survey results of 4 trial products

Trial product	Amount of sugar (feature-1)	Amount of CO_2 (feature-2)	Response (label)
#1	1	4	Dislike
#2	5	1	Dislike
#3	4	7	Like
#4	8	5	Like

Question: What is the response of another trial product with sugar of 5 and CO_2 of 5? 'Like' or 'Dislike'?

Classification example (cont.)

To predict for the query point (5, 5),

1. compute its distances to all examples,
2. and rank the examples according to the distances

- The Euclidean distance, i.e. straight-line distance, between $x = (x_1, x_2)$ and $y = (y_1, y_2)$ is $\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$

  E.g. Distance between the query point and trial product #1

  = distance between (5, 5) and (1, 4)

  $= \sqrt{(5 - 1)^2 + (5 - 4)^2}$

  = 4.12

Classification example (cont.)

Trial product	Amount of sugar (feature-1)	Amount of CO_2 (feature-2)	Response (label)	Distance to (5, 5)	Rank by minimum distance
#1	1	4	Dislike	4.12	4
#2	5	1	Dislike	4.00	3
#3	4	7	Like	2.24	1
#4	8	5	Like	3.00	2

Identify $k=3$ nearest examples

For 3-NN,

- 3 nearest neighbours are #3, #4, and #2 – with labels 'Like', 'Like', and 'Dislike'
- the predicted label for (5, 5) is the majority class, i.e. 'Like'.

What is the predicted label if 1-NN is used?

Regression example

Assume numerical responses (labels) – scores of liking

Survey results of 4 trial products

Trial product	Amount of sugar (feature-1)	Amount of CO_2 (feature-2)	Response (label)
#1	1	4	3.5
#2	5	1	4.6
#3	4	7	5.1
#4	8	5	6.5

Question: What is the response of another trial product with sugar of 5 and CO_2 of 5? What is the particular response score (num.)?

Regression example (cont.)

Same distances and ranks as in the previous example

Trial prod-uct	Amount of sugar (feature-1)	Amount of CO_2 (feature-2)	Response (label)	Distance to (5, 5)	Rank by minimum distance
#1	1	4	3.5	4.12	4
#2	5	1	4.6	4.00	3
#3	4	7	5.1	2.24	1
#4	8	5	6.5	3.00	2

For 3-NN,

- 3 nearest neighbours are #3, #4, and #2 – with labels 5.1, 6.5, and 4.6,
- the predicted label is the mean (5.1+6.5+4.6)/3 = 5.4

What is the predicted label if 1-NN or 2-NN is used?

Weighted k-NN

This basic k-NN algorithm works treats all neighbors equally.

Weighted k-NN introduces the concept of assigning different weights to neighbors based on their proximity to the query point, which can improve performance.

- Closer neighbours have more influence and thus larger weights
- Farther neighbours have less influence and thus smaller weights

E.g. one way of calculating the weight w_i of the i-th neighbour n_i for the query point x is

$$W_i = \frac{e^{-d(x,n_i)}}{\sum_{i=1}^{k} e^{-d(x,n_i)}}$$

Ref: https://medium.com/@lakshmiteja.ip/understanding-weighted-k-nearest-neighbors-k-nn-algorithm-3485001611ce

Higher dimensional dataset: 3-features examples

Example	Feature 1	Feature 2	Feature 3	Label	Distance	Rank
#1	1.5	3.4	2.1	55.4	D	
…	…	…	…	…		

- For the query point (1.1, 1.2, 1.3)

$$\text{Distance D} = \sqrt{(1.1 - 1.5)^2 + (1.2 - 3.4)^2 + (1.3 - 2.1)^2}$$

Higher dimensional dataset: 4-features examples

Example	Feature 1	Feature 2	Feature 3	Feature 4	Label	Distance
#1	1	-3	2	5	55.4	D
…	…	…	…	…	…	

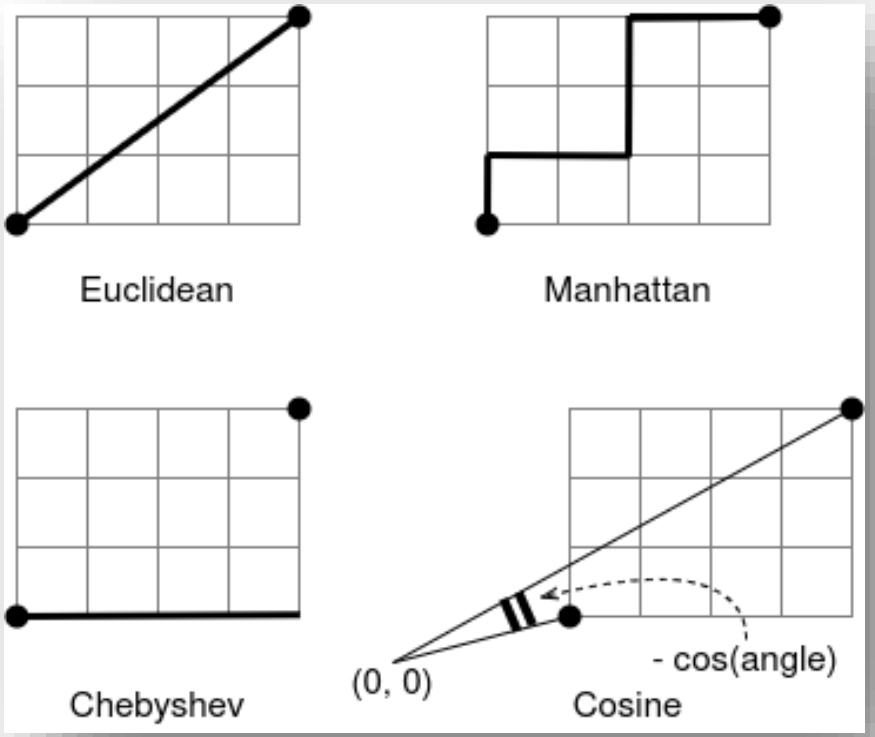
- For the query point $(1, 2, 0, 3)$

  Distance D = $\sqrt{(1 - 1)^2 + (2 - (-3))^2 + (0 - 2)^2 + (3 - 5)^2}$

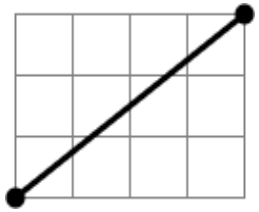
3. Distance metrics

Distance metrics

- Choosing the right distance metric is crucial for k-NN used in classification and regression tasks.
- There are many ways of measuring distances or closeness
- The **Euclidean distance** is the most common



Euclidean distance



The Euclidean distance is the straight-line distance

- For two n-feature examples $x = (x_1, x_2, \ldots, x_n)$ and $y = (y_1, y_2, \ldots, y_n)$

$$\text{Euclidean}(x, y)$$

$$= \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_n - y_n)^2}$$

$$= \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- In 2D, distance between $x = (x_1, x_2)$ and $y = (y_1, y_2)$ is

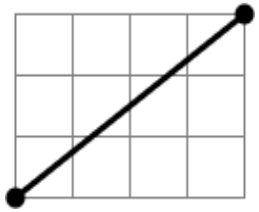
$$\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

E.g.: two data points are (0,0) and (4, 3), Euclidean distance = $\sqrt{(0 - 4)^2 + (0 - 3)^2} = 5$

Euclidean distance (cont.)

Properties

- Sensitive to large differences in feature values.
- Performs well on low-dimensional, normalized data.
- Commonly used for geometric interpretation.



Distance Metric	When to Use	Use Case Scenario
Euclidean Distance	- Continuous numerical data. - When the data is well-scaled.	- Predicting house prices based on square footage and number of bedrooms. - Image recognition where pixel values are continuous features.

Ref: https://www.geeksforgeeks.org/how-to-choose-the-right-distance-metric-in-knn/;
https://www.geeksforgeeks.org/machine-learning/how-to-choose-the-right-distance-metric-in-knn/

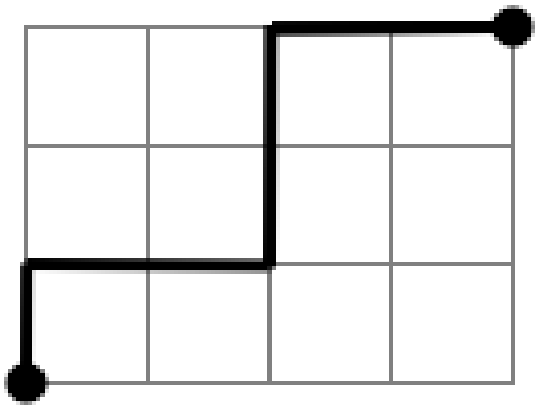
Manhattan distance

The **Manhattan distance** is the sum of the absolute difference across all attributes

- For two n-feature examples $x = (x_1, x_2, \ldots, x_n)$ and $y = (y_1, y_2, \ldots, y_n)$

$$\text{Manhattan}(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

- In 2D, it is the sum of the east-west distance and the north-south distance



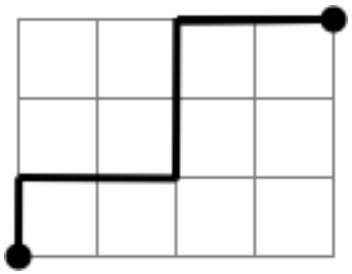
E.g.: two data points are $(0, 0)$ and $(4, 3)$

Manhattan distance = $|0 - 4| + |0 - 3| = 7$

Manhattan distance (cont.)

Properties

- Robust to outliers compared to Euclidean distance.
- Preferred for high-dimensional data.
- Works well in sparse feature environments.



Distance Metric	When to Use	Use Case Scenario
Manhattan Distance	- Data with features on a grid (e.g., city streets). - When data is less sensitive to outliers.	- Delivery routing for trucks following city grids. - Robot navigation through a grid with restricted movement (i.e., only vertical or horizontal). - Infrastructure planning and transportation networks.

Ref: https://www.geeksforgeeks.org/how-to-choose-the-right-distance-metric-in-knn/;
https://www.geeksforgeeks.org/machine-learning/how-to-choose-the-right-distance-metric-in-knn/

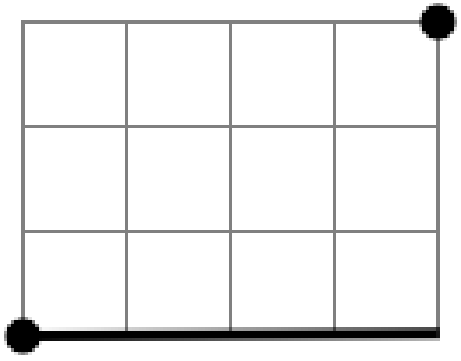
Chebyshev distance

The **Chebyshev distance** is the maximum absolute difference across all attributes

- For two n-feature examples $x = (x_1, x_2, \ldots, x_n)$ and $y = (y_1, y_2, \ldots, y_n)$

$$\text{Chebyshev}(x, y) = \max_{i=1}^{n} | x_i - y_i |$$

- In 2D, it is the maximum of horizontal difference and vertical difference



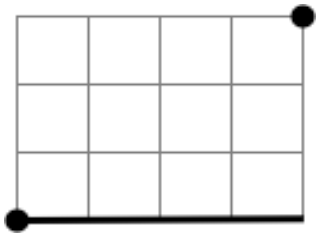
E.g.: two data points are $(0, 0)$ and $(4, 3)$

Chebyshev distance $= \max(|0 - 4|, |0 - 3|) = 4$

Chebyshev distance (cont.)

Properties

- Uses the maximum feature difference
- Ignores smaller variations
- Square-shaped distance boundary



Distance Metric	When to Use	Use Case Scenario
Chebyshev Distance	- When the maximum difference between coordinates is important. - When features represent movements along a grid with equal importance.	- In a board game, measuring the maximum number of moves a piece can make in any direction. - Robot movement where diagonal and straight moves are equally important (e.g., chess, checkers).

Ref: https://www.geeksforgeeks.org/how-to-choose-the-right-distance-metric-in-knn/;
https://www.geeksforgeeks.org/machine-learning/how-to-choose-the-right-distance-metric-in-knn/

Minkowski distance

The **Minkowski distance** is a generalization of the Euclidean, Manhattan, and Chebyshev distances

$$\text{Minkowski}(x,y) = \left(\sum_{i=1}^{n} | x_i - y_i |^p \right)^{\frac{1}{p}}$$

- Also called the ***p*-norm** distance
- The order parameter, p, is called the *norm*
  - The Euclidean distance when p is 2
  - The Manhattan distance when p is 1
  - The Chebyshev distance when p is ∞

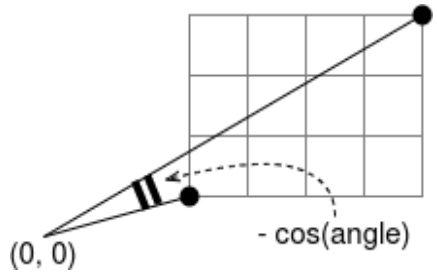
Minkowski distance

The **Minkowski distance** is a generalization of the Euclidean, Manhattan, and Chebyshev distances

Distance Metric	When to Use	Use Case Scenario
Minkowski Distance	- When you need a flexible metric that can represent different distances. - When you want to tune the parameter 'p' for customization.	- Analyzing weather data like temperature, humidity, and wind speed to predict likelihood of rain. - Choosing between Euclidean or Manhattan depending on the problem's spatial relationship.[1][5]

Ref: https://www.geeksforgeeks.org/how-to-choose-the-right-distance-metric-in-knn/;
https://www.geeksforgeeks.org/machine-learning/how-to-choose-the-right-distance-metric-in-knn/

Cosine distance



The **cosine distance** is the negative *cosine similarity*

- The *cosine similarity* is the cosine of the angle between two vectors

Cosine similarity of two vectors x and y

$$\text{Cosine similarity}(x, y) = \frac{x \cdot y}{\|x\| \cdot \|y\|}$$

$$= \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2}\sqrt{\sum_{i=1}^{n} y_i^2}}$$

Range: -1 to 1

- 1: vectors point in the same direction (high similarity)
- 0: vectors are orthogonal (no relation)
- -1: vectors opposite direction (high dissimilarity)

Ref: https://www.geeksforgeeks.org/machine-learning/how-to-choose-the-right-distance-metric-in-knn/

Cosine distance (cont.)

Cosine similarity of two vectors x and y

$$\text{Cosine similarity}(x, y) = \frac{x \cdot y}{\|x\| \cdot \|y\|} = \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2} \sqrt{\sum_{i=1}^{n} y_i^2}}$$

Properties

- Measures angle, not magnitude
- Good for text or high-dimensional data
- Scale-independent

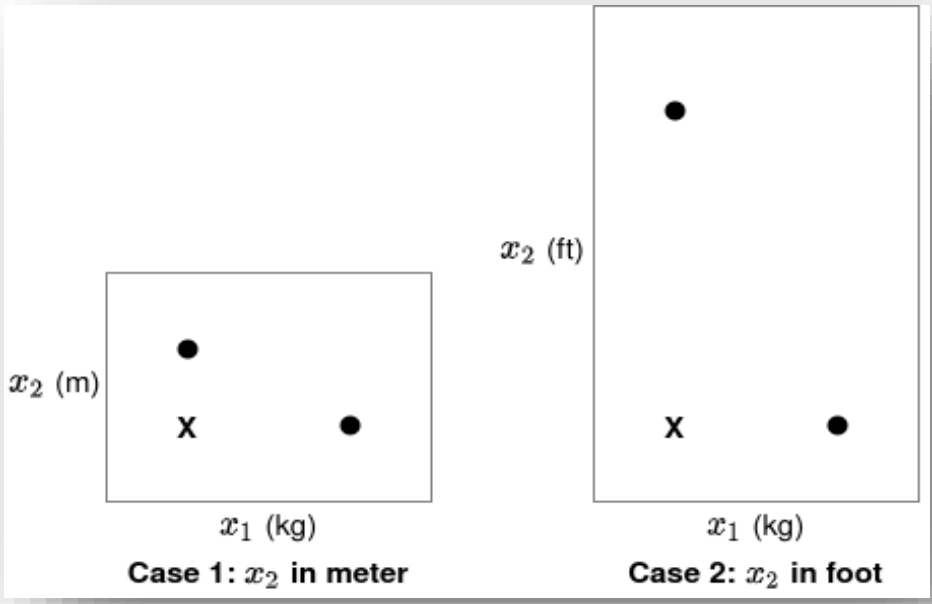
Distance Metric	When to Use	Use Case Scenario
Cosine Similarity	- When the direction of the vectors is more important than their magnitude.	- Text analysis, image retrieval, and recommendation systems. [Note: Cosine similarity is not a distance metric but a similarity measure, yet it is often used in similar contexts.]

Ref: https://www.geeksforgeeks.org/how-to-choose-the-right-distance-metric-in-knn/;
https://www.geeksforgeeks.org/machine-learning/how-to-choose-the-right-distance-metric-in-knn/

4. Characteristics of *k*-NN

Data scaling is needed in kNN

Consider a dataset represented in two forms: height (x_2) in meter and in foot. Which dot (neighbour) is closer to 'X' (query point)?



- Scaling the data is an important preprocessing step for the KNN algorithm because the algorithm is distance-based.
- If the data is not scaled, features with larger scales will dominate the distance calculation and can result in incorrect predictions.

Data/feature scaling

Feature scaling commonly uses two techniques: Normalization and Standardization. There are several methods under each of these categories, but two of the most common ones are:

1. Min-max scaling: This technique scales the data to a specific range (usually between 0 and 1) or [-1,1] or [0,5] etc. Min-max scaling preserves the original distribution of the data but may be sensitive to outliers. The formula for min-max scaling is given below:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

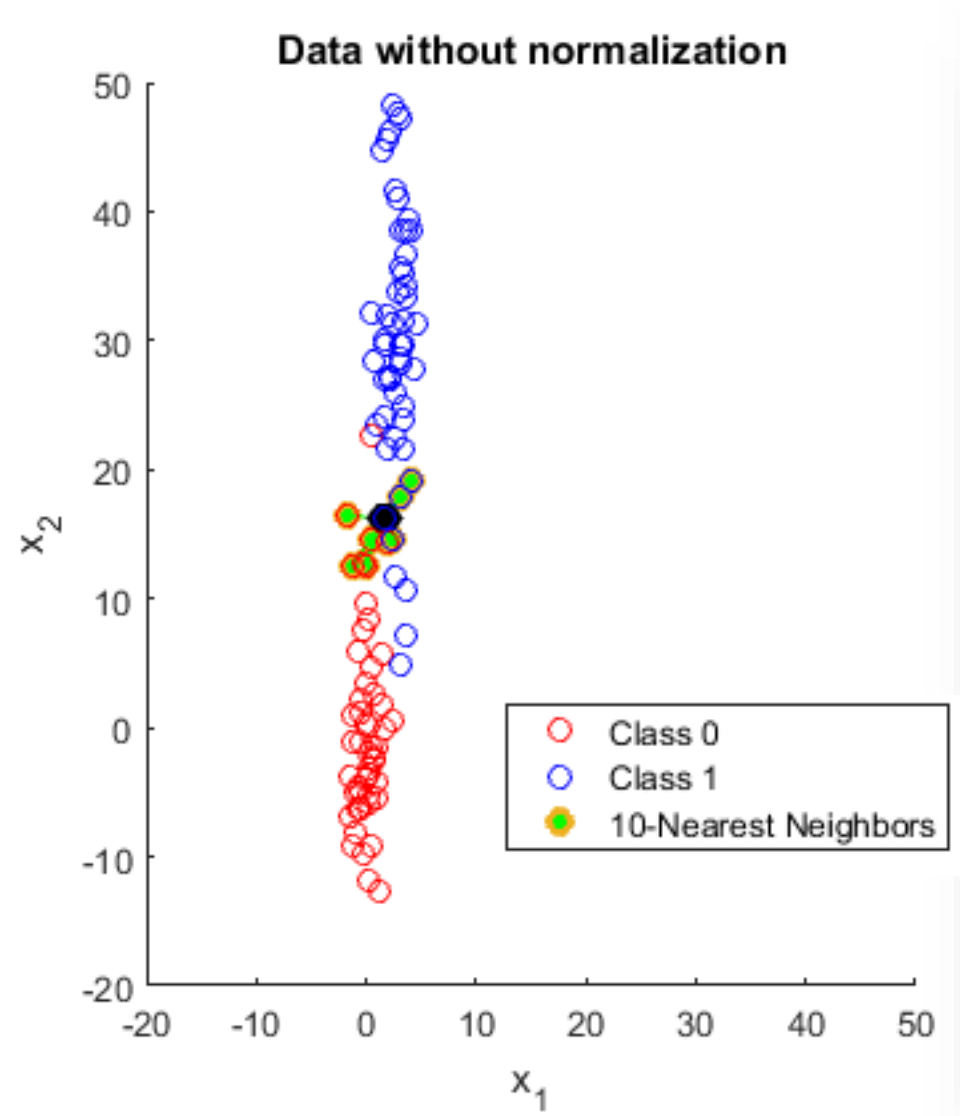
2. Standardization: This technique scales the data to have zero mean and unit variance. Standardization is robust to outliers but may change the distribution of the data. The standardization formula is given below:

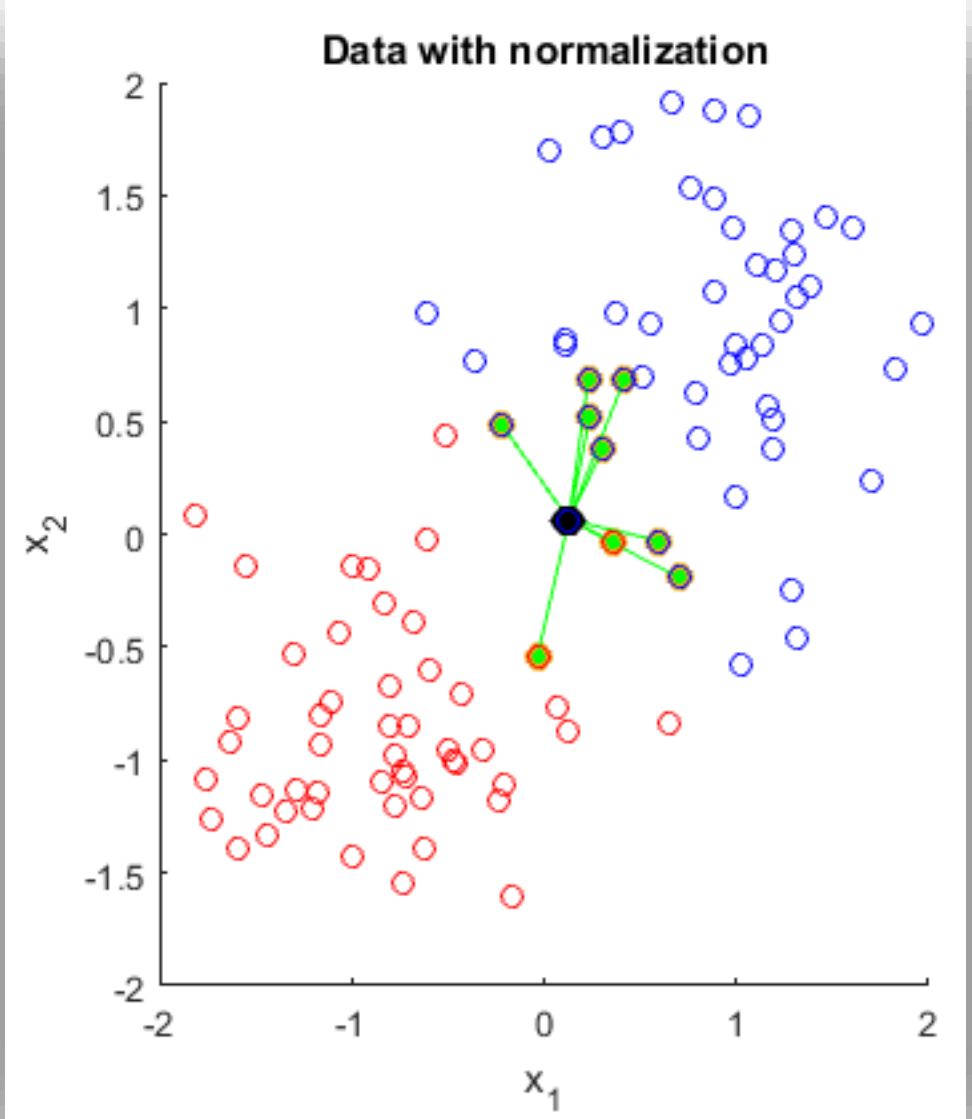
$$X_{scaled} = \frac{X - X_{mean}}{X_{stddev}}$$

- Scaling the features can improve the accuracy of the KNN algorithm and ensure that it's robust to changes in the scale of the data.

Ref: https://aiml.com/what-is-feature-scaling-explain-the-different-feature-scaling-techniques/

Without data scaling, all the nearest neighbors are aligned in the direction of the axis with the smaller range, i.e., x_1 leading to incorrect classification/prediction. Normalization solves this problem!





Source: https://stats.stackexchange.com/questions/287425/why-do-you-need-to-scale-data-in-knn

The value of *k* in *kNN*

- Commonly-used values of ***k* are 1, 3, 10, and 20**

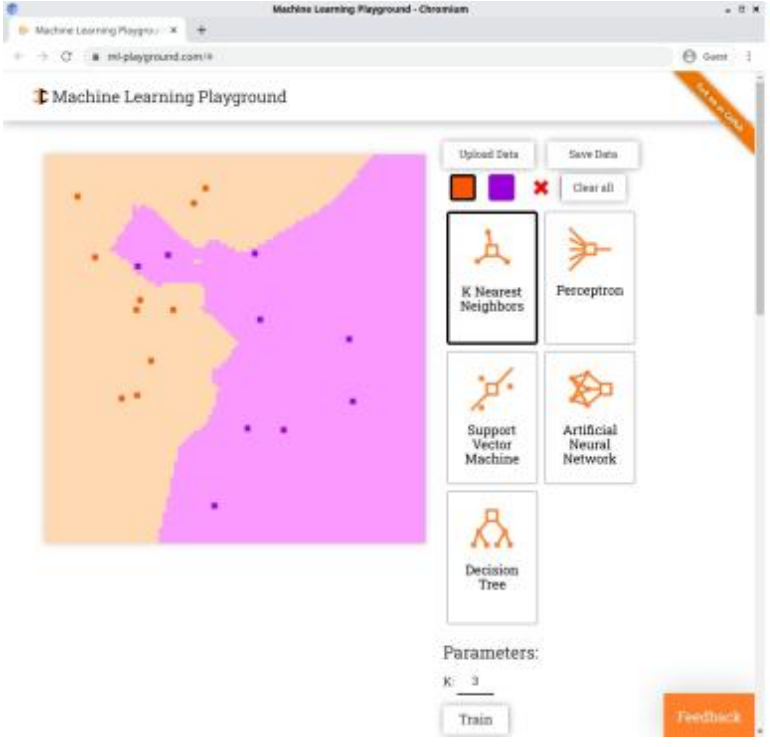
  Another suggestion: for a dataset of n examples, consider **values of *k* from $\sqrt{n}$ to 1**

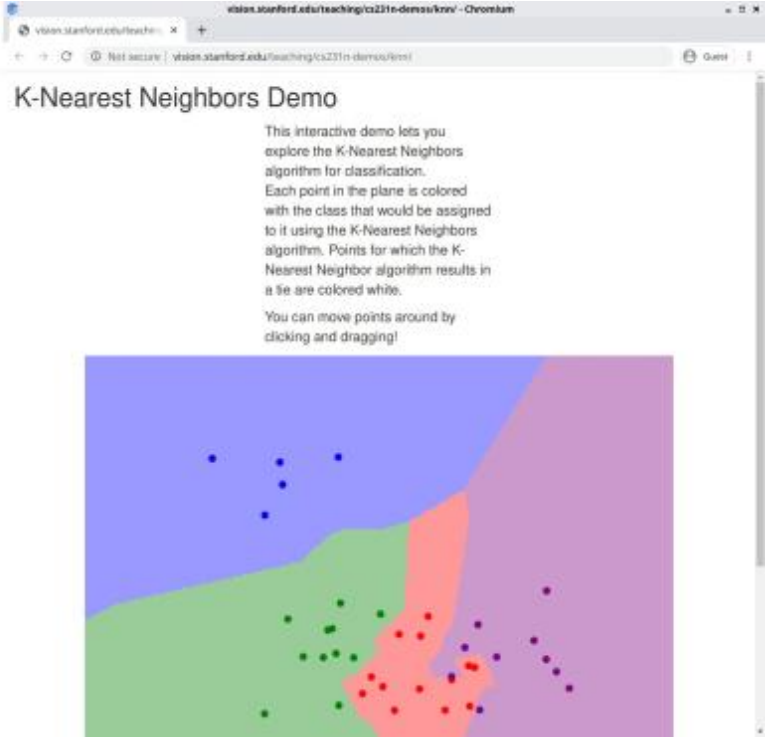
- When *k* is small,
  - *k*-NN depends on the very specific details of the training data
  - and is severely affected by noise (*overfitting* as you'll learn later)

- When *k* is large,
  - *k*-NN depends on the average of most training data
  - and is barely affected by noise (*underfitting* as you'll learn later)

Decision boundaries in *k*-NN

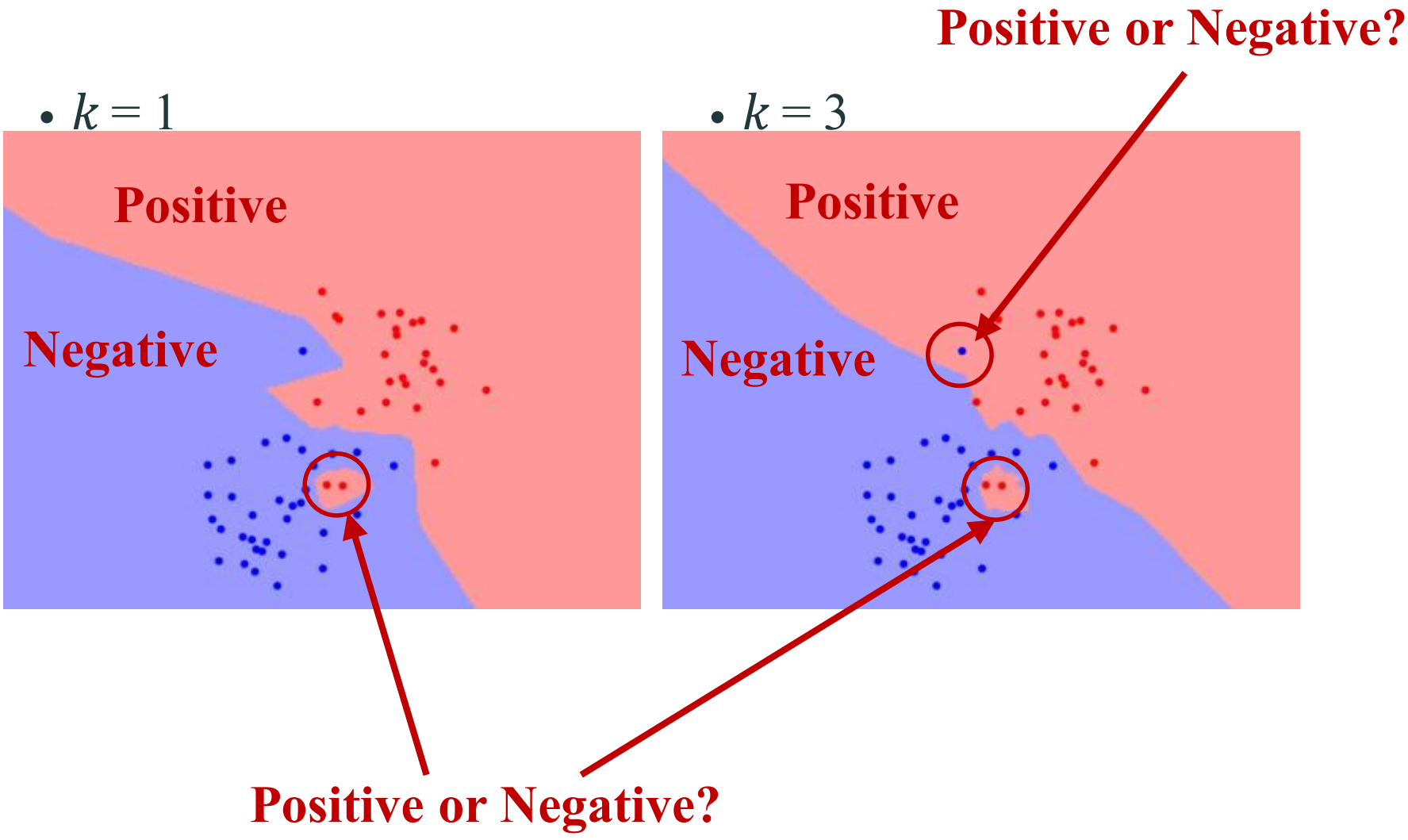
- Decision boundary is an imaginary line or surface that separates different classes in a classification problem.
- Decision boundaries in kNN are directly influenced by the number of neighbors "K" and the spatial distribution of data points in training set.



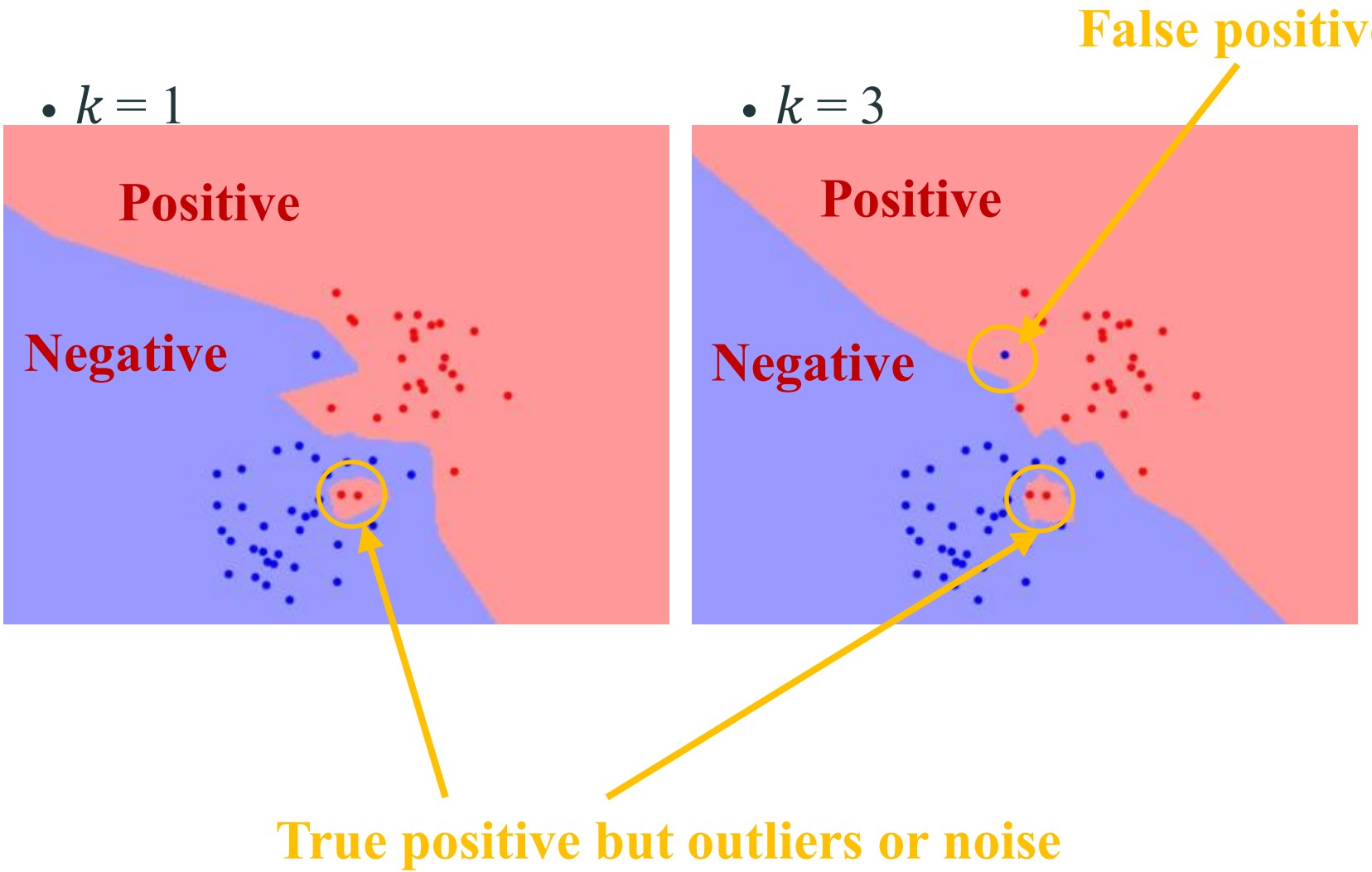


https://ml-playground.com/ http://vision.stanford.edu/teaching/cs231n-demos/knn/

Decision boundary: $k = 1, k = 3$

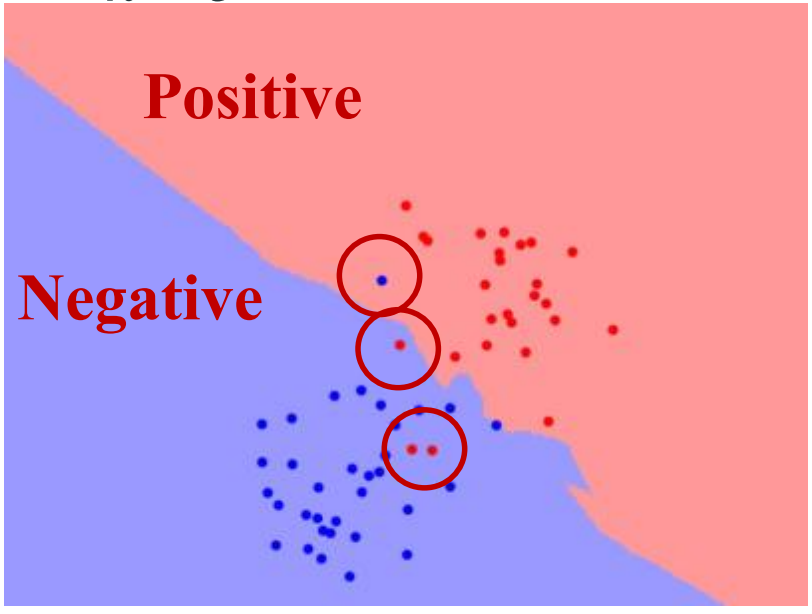


Decision boundary: $k = 1, k = 3$

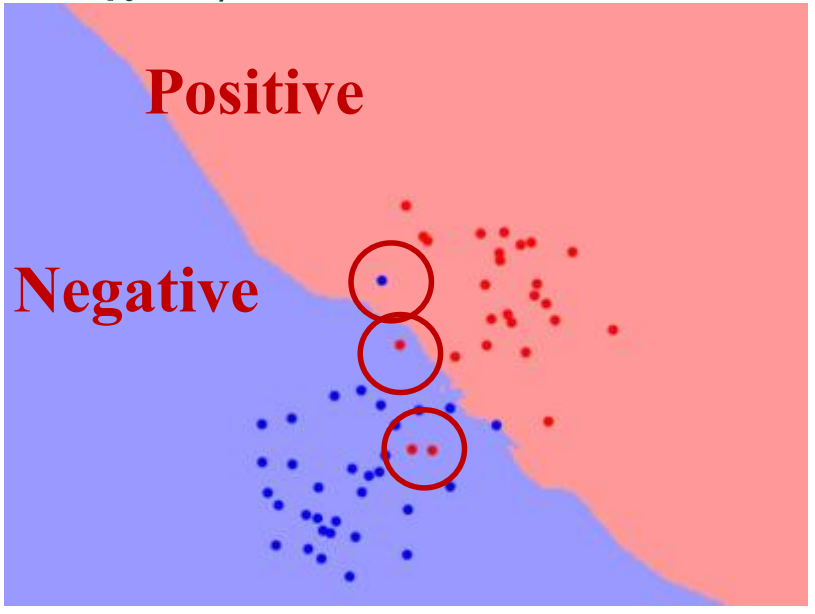


Decision boundary: $k=5, k=7$

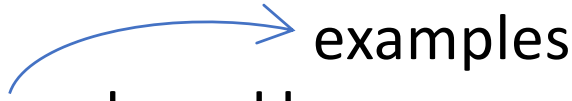
- $k=5$



- $k=7$



k-NN is lazy and non-parametric



k-NN is a lazy learner or instance-based learner

- It doesn't learn at all during training, but memorizes (stores) the examples for later use
- It is fast in training, but slow in predicting, esp. for huge datasets
- Linear and logistic regression are *eager learners*

k-NN is a non-parametric method

- It doesn't assume a relation between features and labels
- Linear and logistic regression are *parametric* methods, assuming a linear relation

Pros and cons of *k*-NN

+ Generally effective and accurate with sufficient training data

+ Robust to noise in training data (for proper *k*)

+ Ready for online learning, i.e. when example data are added during production

- Need to determine the optimal distance metric and features to use (may require domain knowledge)

- High computation cost for calculating distances to find nearest neighbours

Summary of *k*-NN

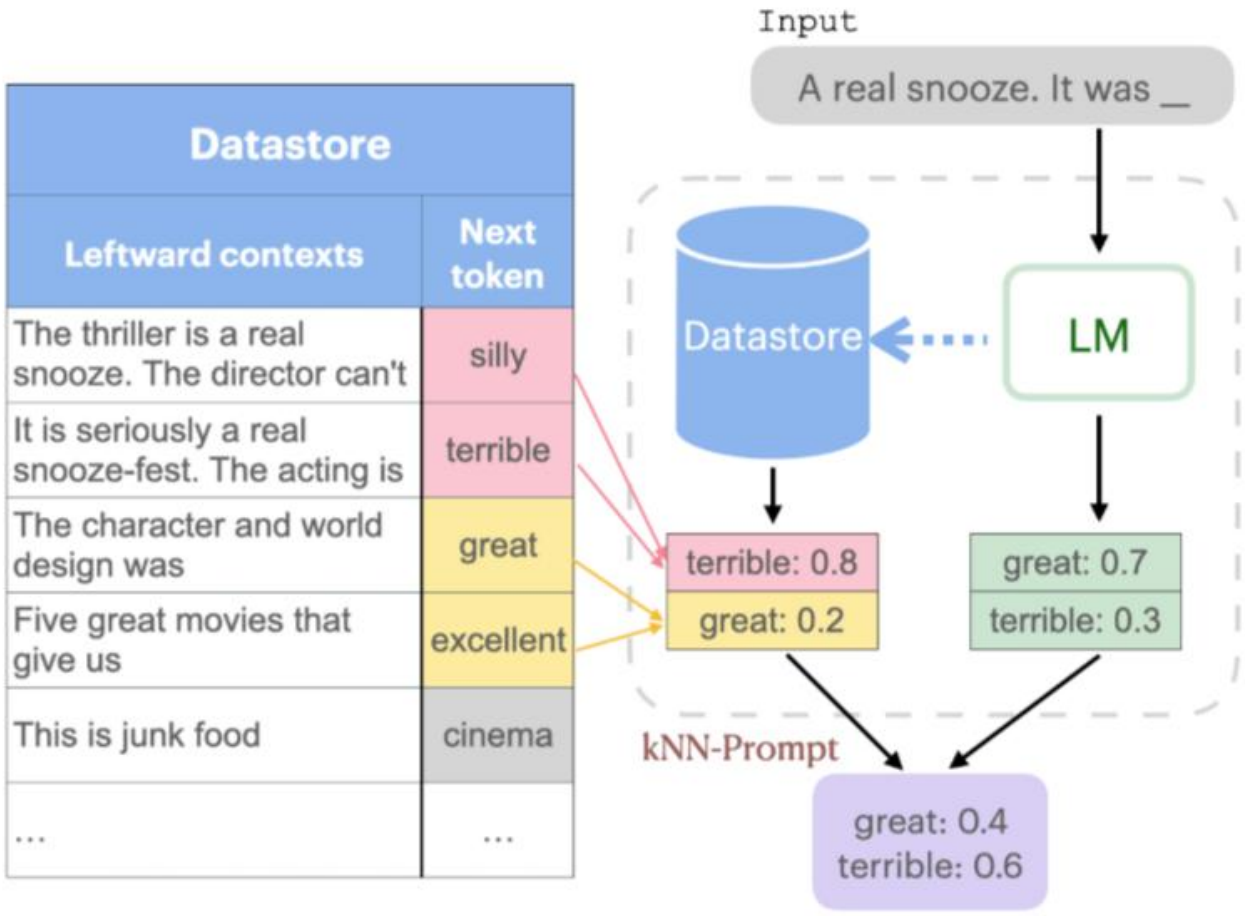
- Easy to implement: Low complexity compared to other machine learning algorithms.
- No training required: KNN stores all data in memory and doesn't require any training, it automatically adjusts and uses the new data for future predictions.
- Few Hyperparameters: The only parameters in the training are the value of k and the choice of the distance metric.
- Flexible: It works for classification problems and regression tasks.
- Doesn't scale well: It is very slow especially with large datasets.
- Prone to Overfitting: As the algorithm is affected due to the curse of dimensionality it is prone to the problem of overfitting as well.

Ref: *https://www.geeksforgeeks.org/k-nearest-neighbours/*

Appendix (Not Examined)

k-NN + AIGC

K-Nearest Neighbor (KNN) Prompting



Learn Prompting

Ref: https://learnprompting.org/docs/advanced/few_shot/k_nearest_neighbor_knn?srsltid=AfmBOoqXQUEj5sFotTQgRfKFV9YfBPZ6CLnQRtpgnMbFcurcqRc9YJpg

The end